



Walmart Sales Analysis using SQL

Mentorless Internship Task 2

-Nevil Hitesh Mehta



Precautions before Importing Data

1. Remove any blank rows/columns.
2. Renaming of columns to lowercase, avoid using special characters, keywords.
3. Formatting the Data-type of every variable.
4. Remove comma used as a thousand's separator if present.

Original Data

Invoice_ID	Branch	City	Customer_type	Gender	Product_line	Unit_price	Quantity	Tax_5%	Total	Date	Time	Payment
750-67-8428	A	Yangon	Member	Female	Health and beau	74.69	7	26.1415	548.9715	05-01-2019	13:08:00	Ewallet
226-31-3081	C	Naypyitaw	Normal	Female	Electronic acces	15.28	5	3.82	80.22	08-03-2019	10:29:00	Cash
631-41-3108	A	Yangon	Normal	Male	Home and lifesty	46.33	7	16.2155	340.5255	03-03-2019	13:23:00	Credit card
123-19-1176	A	Yangon	Member	Male	Health and beau	58.22	8	23.288	489.048	27-01-2019	20:33:00	Ewallet
373-73-7910	A	Yangon	Normal	Male	Sports and trave	86.31	7	30.2085	634.3785	08-02-2019	10:37:00	Ewallet
699-14-3026	C	Naypyitaw	Normal	Male	Electronic acces	85.39	7	29.8865	627.6165	25-03-2019	18:30:00	Ewallet
355-53-5943	A	Yangon	Member	Female	Electronic acces	68.84	6	20.652	433.692	25-02-2019	14:36:00	Ewallet
315-22-5665	C	Naypyitaw	Normal	Female	Home and lifesty	73.56	10	36.78	772.38	24-02-2019	11:38:00	Ewallet
665-32-9167	A	Yangon	Member	Female	Health and beau	36.26	2	3.626	76.146	10-01-2019	17:15:00	Credit card
692-92-5582	B	Mandalay	Member	Female	Food and bevera	54.84	3	8.226	172.746	20-02-2019	13:27:00	Credit card
351-62-0822	B	Mandalay	Member	Female	Fashion accesssc	14.48	4	2.896	60.816	06-02-2019	18:07:00	Ewallet
529-56-3974	B	Mandalay	Member	Male	Electronic acces	25.51	4	5.102	107.142	09-03-2019	17:03:00	Cash
365-64-0515	A	Yangon	Normal	Female	Electronic acces	46.95	5	11.7375	246.4875	12-02-2019	10:25:00	Ewallet
252-56-2699	A	Yangon	Normal	Male	Food and bevera	43.19	10	21.595	453.495	07-02-2019	16:48:00	Ewallet
829-34-3910	A	Yangon	Normal	Female	Health and beau	71.38	10	35.69	749.49	29-03-2019	19:21:00	Cash
299-46-1805	B	Mandalay	Member	Female	Sports and trave	93.72	6	28.116	590.436	15-01-2019	16:19:00	Cash
656-95-9349	A	Yangon	Member	Female	Health and beau	68.93	7	24.1255	506.6355	11-03-2019	11:03:00	Credit card
765-26-6951	A	Yangon	Normal	Male	Sports and trave	72.61	6	21.783	457.443	01-01-2019	10:39:00	Credit card
329-62-1586	A	Yangon	Normal	Male	Food and bevera	54.67	3	8.2005	172.2105	21-01-2019	18:00:00	Credit card
319-50-3348	B	Mandalay	Normal	Female	Home and lifesty	40.3	2	4.03	84.63	11-03-2019	15:30:00	Ewallet

Modified Data

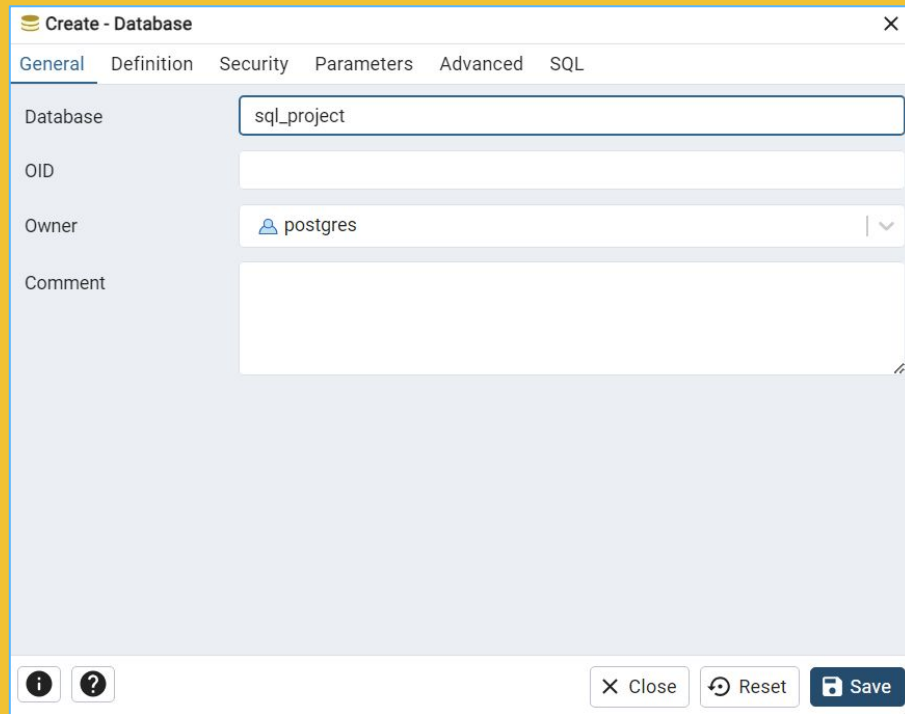
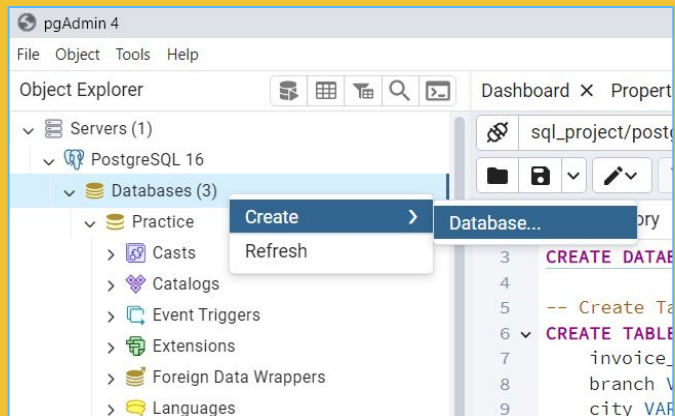
invoice_id	branch	city	customer_type	gender	product_line	unit_price	quantity	tax_5percent	purchase_total	purchase_date	purchase_time	payment
750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.14	548.97	2019-05-01	13:08:00	Ewallet
226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82	80.22	2019-08-03	10:29:00	Cash
631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.22	340.53	2019-03-03	13:23:00	Credit card
123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.29	489.05	2019-01-27	20:33:00	Ewallet
373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.21	634.38	2019-08-02	10:37:00	Ewallet
699-14-3026	C	Naypyitaw	Normal	Male	Electronic accessories	85.39	7	29.89	627.62	2019-03-25	18:30:00	Ewallet
355-53-5943	A	Yangon	Member	Female	Electronic accessories	68.84	6	20.65	433.69	2019-02-25	14:36:00	Ewallet
315-22-5665	C	Naypyitaw	Normal	Female	Home and lifestyle	73.56	10	36.78	772.38	2019-02-24	11:38:00	Ewallet
665-32-9167	A	Yangon	Member	Female	Health and beauty	36.26	2	3.63	76.15	2019-10-01	17:15:00	Credit card
692-92-5582	B	Mandalay	Member	Female	Food and beverages	54.84	3	8.23	172.75	2019-02-20	13:27:00	Credit card
351-62-0822	B	Mandalay	Member	Female	Fashion accessories	14.48	4	2.90	60.82	2019-06-02	18:07:00	Ewallet
529-56-3974	B	Mandalay	Member	Male	Electronic accessories	25.51	4	5.10	107.14	2019-09-03	17:03:00	Cash
365-64-0515	A	Yangon	Normal	Female	Electronic accessories	46.95	5	11.74	246.49	2019-12-02	10:25:00	Ewallet
252-56-2699	A	Yangon	Normal	Male	Food and beverages	43.19	10	21.60	453.50	2019-07-02	16:48:00	Ewallet
829-34-3910	A	Yangon	Normal	Female	Health and beauty	71.38	10	35.69	749.49	2019-03-29	19:21:00	Cash
299-46-1805	B	Mandalay	Member	Female	Sports and travel	93.72	6	28.12	590.44	2019-01-15	16:19:00	Cash
656-95-9349	A	Yangon	Member	Female	Health and beauty	68.93	7	24.13	506.64	2019-11-03	11:03:00	Credit card
765-26-6951	A	Yangon	Normal	Male	Sports and travel	72.61	6	21.78	457.44	2019-01-01	10:39:00	Credit card
329-62-1586	A	Yangon	Normal	Male	Food and beverages	54.67	3	8.20	172.21	2019-01-21	18:00:00	Credit card
319-50-3348	B	Mandalay	Normal	Female	Home and lifestyle	40.30	2	4.03	84.63	2019-11-03	15:30:00	Ewallet



Importing Data into SQL

1. Create Database
2. Create Table
3. Import Data

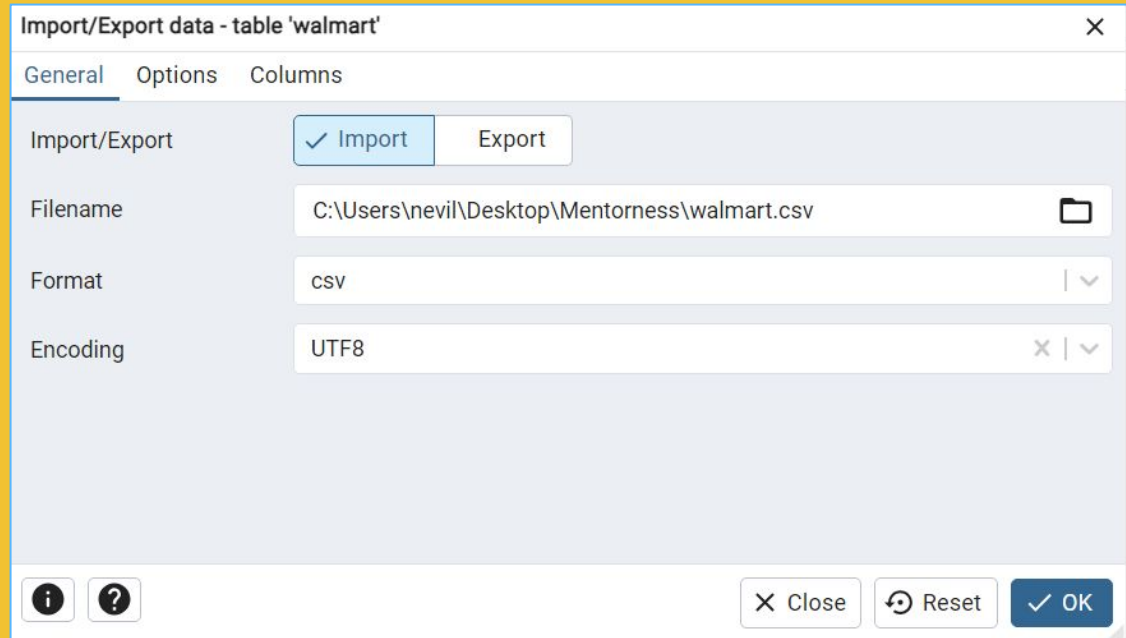
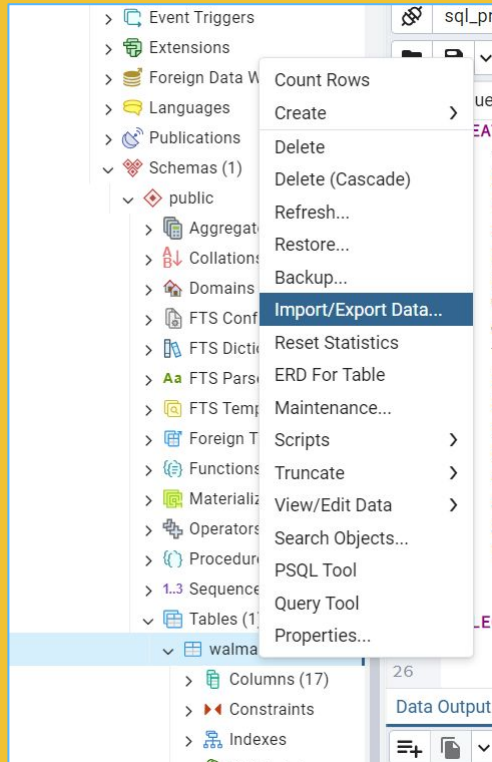
Creating Database



Creating Table

```
2
3  -- Create Table called "walmart"
4  CREATE TABLE walmart(
5      invoice_id VARCHAR(15) PRIMARY KEY,
6      branch VARCHAR(2),
7      city VARCHAR(15),
8      customer_type VARCHAR(10),
9      gender VARCHAR(10),
10     product_line VARCHAR(20),
11     unit_price FLOAT,
12     quantity INT,
13     tax_5percent FLOAT,
14     purchase_total FLOAT,
15     purchase_date DATE,
16     purchase_time TIME,
17     payment VARCHAR(15),
18     cogs FLOAT,
19     gross_margin_percentage FLOAT,
20     gross_income FLOAT,
21     rating FLOAT
22 );
```

Importing Data-1



Importing Data-2

Import/Export data - table 'walmart' ✕

General Options Columns

OID ☐

Header ☒

Delimiter

,

⌵

Specifies the character that separates columns within each row (line) of the file. The default is a tab character in text format, a comma in CSV format. This must be a single one-byte character. This option is not allowed when using binary format.

Quote

"

✕ ⌵

Specifies the quoting character to be used when a data value is quoted.

?

?

✕ Close

↺ Reset

✓ OK

Import/Export data - table 'walmart' ✕

General Options Columns

Columns to import

invoice_id ✕ branch ✕ city ✕ customer_type ✕ gender ✕

product_line ✕ unit_price ✕ quantity ✕ tax_5percent ✕

purchase_total ✕ purchase_date ✕ purchase_time ✕

payment ✕ cogs ✕ gross_margin_percentage ✕

gross_income ✕ rating ✕

⌵

An optional list of columns to be copied. If no column list is specified, all columns of the table will be copied.

NOT NULL columns

Not null columns... ⌵

Do not match the specified column values against the null string. In the

?

?

✕ Close

↺ Reset

✓ OK

9

Importing Data-3

24 `SELECT * FROM walmart;`
25

Data Output Messages Notifications

	invoice_id [PK] character varying (15)	branch character varying (2)	city character varying (15)	customer_type character varying (10)	gender character varying (10)	product_line character varying (25)	unit_price double precision	quantity integer	tax_5percent double precision	pu doi
1	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.14	
2	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82	
3	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.22	
4	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.29	
5	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.21	
6	699-14-3026	C	Naypyitaw	Normal	Male	Electronic accessories	85.39	7	29.89	
7	355-53-5943	A	Yangon	Member	Female	Electronic accessories	68.84	6	20.65	

Total rows: 1000 of 1000 Query complete 00:00:00.224 Ln 24, Col 1



Reviewing Data

1. Check for completeness of data.
2. Checking for inconsistencies : null values
3. Exploring the data thoroughly.

Completeness Check

```
30 -- Check for total no.of rows present
31
32 SELECT COUNT(*) FROM walmart;
33
```

Data Output Messages Notifications



	count	
	bigint	🔒
1	1000	

Inconsistency Check

The screenshot shows a SQL IDE interface with a query editor and a data output pane. The query is as follows:

```
28 SELECT * FROM walmart
29 WHERE
30     invoice_id IS NULL
31     OR
32     branch IS NULL
33     OR
34     city IS NULL
35     OR
36     customer_type IS NULL
37     OR
38     gender IS NULL
39     OR
40     product_line IS NULL
41     OR
42     unit_price IS NULL
43     OR
44     quantity IS NULL
45     OR
46     tax_5percent IS NULL
47     OR
48     purchase_total IS NULL
```

The Data Output pane shows the following columns and their data types:

Invoice_Id	branch	city	customer_type	gender	product_line	unit_price	quantity
[PK] character varying (15)	character varying (2)	character varying (15)	character varying (10)	character varying (10)	character varying (25)	double precision	integer

The screenshot shows the same SQL IDE interface, but with the query executed. The Data Output pane shows the following results:

```
28 SELECT * FROM walmart
29 WHERE
30     invoice_id IS NULL
31     OR
32     branch IS NULL
33     OR
34     city IS NULL
35     OR
36     customer_type IS NULL
37     OR
38     gender IS NULL
39     OR
40     product_line IS NULL
41     OR
42     unit_price IS NULL
43     OR
44     quantity IS NULL
45     OR
46     tax_5percent IS NULL
47     OR
48     purchase_total IS NULL
```

The Data Output pane shows the following columns and their data types:

Invoice_Id	branch	city	customer_type	gender	product_line	unit_price	quantity
[PK] character varying (15)	character varying (2)	character varying (15)	character varying (10)	character varying (10)	character varying (25)	double precision	integer

Successfully run. Total query runtime: 164 msec.
9 rows affected.

Exploring Data-1

```
75 -- Unique Branches
76 SELECT COUNT(DISTINCT(branch)) AS total_branches FROM walmart;
77
```

Data Output Messages Notifications

	total_branches bigint
1	3

```
79 -- Exploring Cities
80 SELECT COUNT(DISTINCT(city)) AS total_cities FROM walmart;
81 SELECT DISTINCT(city) AS cities FROM walmart;
```

Data Output Messages Notifications

	total_cities bigint
1	3

```
75 -- Unique Branches
76 SELECT COUNT(DISTINCT(branch)) AS total_branches FROM walmart;
77 SELECT DISTINCT(branch) AS branches FROM walmart;
78
79
80
```

Data Output Messages Notifications

	branches character varying (2)
1	A
2	B
3	C

```
79 -- Exploring Cities
80 SELECT COUNT(DISTINCT(city)) AS total_cities FROM walmart;
81 SELECT DISTINCT(city) AS cities FROM walmart;
82
83
```

Data Output Messages Notifications

	cities character varying (15)
1	Yangon
2	Naypyitaw
3	Mandalay

Exploring Data-2

```
83 -- Exploring Customer
84 SELECT COUNT(DISTINCT(customer_type)) AS total_customers FROM walmart;
85 SELECT DISTINCT(customer_type) AS customers FROM walmart;
```

Data Output Messages Notifications

total_customers bigint	
1	2

```
87 -- Exploring Gender
88 SELECT COUNT(DISTINCT(gender)) AS total_genders FROM walmart;
89 SELECT DISTINCT(gender) AS genders FROM walmart;
```

Data Output Messages Notifications

total_genders bigint	
1	2

```
83 -- Exploring Customer
84 SELECT COUNT(DISTINCT(customer_type)) AS total_customers FROM walmart;
85 SELECT DISTINCT(customer_type) AS customers FROM walmart;
```

Data Output Messages Notifications

customers character varying (10)	
1	Normal
2	Member

```
87 -- Exploring Gender
88 SELECT COUNT(DISTINCT(gender)) AS total_genders FROM walmart;
89 SELECT DISTINCT(gender) AS genders FROM walmart;
```

Data Output Messages Notifications

genders character varying (10)	
1	Female
2	Male

Exploring Data-3

```
91 -- Exploring Product_Lines
92 SELECT COUNT(DISTINCT(product_line)) AS total_product_lines FROM walmart;
93 SELECT DISTINCT(product_line) AS product_lines FROM walmart;
```

Data Output Messages Notifications



	total_product_lines
	bigint
1	6

```
91 -- Exploring Product_Lines
92 SELECT COUNT(DISTINCT(product_line)) AS total_product_lines FROM walmart;
93 SELECT DISTINCT(product_line) AS product_lines FROM walmart;
94
```

Data Output Messages Notifications



	product_lines
	character varying (25)
1	Fashion accessories
2	Electronic accessories
3	Health and beauty
4	Food and beverages
5	Sports and travel
6	Home and lifestyle

```
132 -- Exploring Payment_modes
133 SELECT COUNT(DISTINCT(payment)) AS total_payment_modes FROM walmart;
134 SELECT DISTINCT(payment) AS payment_modes FROM walmart;
```

Data Output Messages Notifications



	total_payment_modes
	bigint
1	3

```
132 -- Exploring Payment_modes
133 SELECT COUNT(DISTINCT(payment)) AS total_payment_modes FROM walmart;
134 SELECT DISTINCT(payment) AS payment_modes FROM walmart;
135
```

Data Output Messages Notifications



	payment_modes
	character varying (15)
1	Credit card
2	Ewallet
3	Cash

Exploring Data-4

```
97  -- Exploring unit_price
98  CREATE VIEW unit_price_summ AS
99  SELECT
100     MIN(unit_price) AS min_unit_price,
101     MAX(unit_price) AS max_unit_price,
102     round(AVG(unit_price::numeric),2) AS avg_unit_price
103  FROM walmart;
104
105  SELECT * FROM unit_price_summ;
```

Data Output Messages Notifications

SQL

	min_unit_price double precision	max_unit_price double precision	avg_unit_price numeric
1	10.08	99.96	55.67

```
105  -- Exploring Quantity
106  CREATE VIEW quantity_summary AS
107  SELECT
108     MIN(quantity) AS min_quantity,
109     MAX(quantity) AS max_quantity
110  FROM walmart;
111
112  SELECT * FROM quantity_summary;
```

Data Output Messages Notifications

SQL

	min_quantity integer	max_quantity integer
1	1	10

Exploring Data-5

```
114 -- Exploring purchase_date
115 CREATE VIEW date_summary AS
116 SELECT
117     MIN(purchase_date) AS start_date,
118     MAX(purchase_date) AS end_date
119 FROM walmart;
120
121 SELECT * FROM date_summary;
```

Data Output Messages Notifications

	start_date date	end_date date
1	2019-01-01	2019-12-03

```
122
123 -- Exploring purchase_time
124 CREATE VIEW time_summary AS
125 SELECT
126     MIN(purchase_time) AS start_time,
127     MAX(purchase_time) AS end_time
128 FROM walmart;
129
130 SELECT * FROM time_summary;
```

Data Output Messages Notifications

	start_time time without time zone	end_time time without time zone
1	10:00:00	20:59:00

Exploring Data-6

```
138 -- Exploring purchase_total
139 CREATE VIEW purchase_total_summ AS
140 SELECT
141     MIN(purchase_total) AS min_purchase,
142     MAX(purchase_total) AS max_purchase,
143     round(AVG(purchase_total::numeric),2) AS avg_purchase_total
144 FROM walmart;
145
146 SELECT * FROM purchase_total_summ
```

Data Output Messages Notifications

SQL

	min_purchase double precision	max_purchase double precision	avg_purchase_total numeric
1	10.68	1042.65	322.97

```
148 -- Exploring cogs
149 CREATE VIEW cogs_summ AS
150 SELECT
151     MIN(cogs) AS min_cogs,
152     MAX(cogs) AS max_cogs,
153     round(AVG(cogs::numeric),2) AS avg_cogs
154 FROM walmart;
155
156 SELECT * FROM cogs_summ
```

Data Output Messages Notifications

SQL

	min_cogs double precision	max_cogs double precision	avg_cogs numeric
1	10.17	993	307.59

Exploring Data-7

```
158 -- Exploring gross_income
159 CREATE VIEW income_summ AS
160 SELECT
161     MIN(gross_income) AS min_income,
162     MAX(gross_income) AS max_income,
163     round(AVG(gross_income::numeric),2) AS avg_income
164 FROM walmart;
165
166 ▾ SELECT * FROM income_summ
```

Data Output Messages Notifications

	min_income double precision	max_income double precision	avg_income numeric
1	0.51	49.65	15.38

```
168 -- Exploring rating_summary
169 CREATE VIEW rating_summ AS
170 SELECT
171     MIN(rating) AS min_rating,
172     MAX(rating) AS max_rating,
173     round(AVG(rating::numeric),2) AS avg_rating
174 FROM walmart;
175
176 ▾ SELECT * FROM rating_summ
177
```

Data Output Messages Notifications

	min_rating double precision	max_rating double precision	avg_rating numeric
1	4	10	6.97

Sales Analysis-1

```
177 -- 1. Retrieve all columns for sales made in a specific branch (e.g., Branch 'A').
178
179
180 SELECT * FROM walmart
181 WHERE branch = 'A'
182
```

	invoice_id [PK] character varying (15)	branch character varying (2)	city character varying (15)	customer_type character varying (10)	gender character varying (10)	product_line character varying (25)	unit_price double precision	quantity integer
1	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	
5	355-53-5943	A	Yangon	Member	Female	Electronic accessories	68.84	
6	665-32-9167	A	Yangon	Member	Female	Health and beauty	36.26	
7	365-64-0515	A	Yangon	Normal	Female	Electronic accessories	46.95	

Total rows: 340 of 340 Query complete 00:00:00.177 Ln 180, Col 1

```
182
183 -- 2. Find the total sales for each product line.
184
185 SELECT product_line,
186         round(SUM(purchase_total::numeric),2) as total_sales
187 FROM walmart
188 GROUP BY product_line
```

	product_line character varying (25)	total_sales numeric
1	Fashion accessories	54306.03
2	Electronic accessories	54337.64
3	Health and beauty	49193.84
4	Food and beverages	56144.96
5	Sports and travel	55123.00
6	Home and lifestyle	53861.96

Sales Analysis-2

```
190 -- 3. List all sales transactions where the payment method was 'Cash'.
191
192 SELECT *
193 FROM walmart
194 WHERE payment = 'Cash'
```

	product_line character varying (25)	unit_price double precision	quantity integer	tax_5percent double precision	purchase_total double precision	purchase_date date	purchase_time time without time zone	payment character varying (15)	cogs double precision
1	Electronic accessories	15.28	5	3.82	80.22	2019-08-03	10:29:00	Cash	
2	Electronic accessories	25.51	4	5.1	107.14	2019-09-03	17:03:00	Cash	1
3	Health and beauty	71.38	10	35.69	749.49	2019-03-29	19:21:00	Cash	
4	Sports and travel	93.72	6	28.12	590.44	2019-01-15	16:19:00	Cash	5
5	Fashion accessories	33.52	1	1.68	35.2	2019-08-02	15:31:00	Cash	
6	Food and beverages	88.36	5	22.09	463.89	2019-01-25	19:48:00	Cash	
7	Health and beauty	24.89	9	11.2	235.21	2019-03-15	15:36:00	Cash	2
8	Sports and travel	78.07	9	35.13	737.76	2019-01-28	12:43:00	Cash	7

Total rows: 344 of 344 Query complete 00:00:00.148 Ln 190, Col 1

```
196 -- 4. Calculate the total gross income generated in each city.
197
198 SELECT city,
199        round(SUM(gross_income::numeric),2)
200 FROM walmart
201 GROUP BY city
```

city character varying (15)	round numeric
Yangon	5057.36
Naypyitaw	5265.33
Mandalay	5057.36

Sales Analysis-3

```
203 -- 5. Find the average rating given by customers in each branch.
204
205 SELECT branch,
206         round(AVG(rating::numeric),2)
207 FROM walmart
208 GROUP BY branch
209
```

Data Output Messages Notifications

SQL

	branch character varying (2)	round numeric
1	A	7.03
2	B	6.82
3	C	7.07

```
210 -- 6. Determine the total quantity of each product line sold.
211
212 SELECT product_line,
213         SUM(quantity)
214 FROM walmart
215 GROUP BY product_line
```

Data Output Messages Notifications

SQL

	product_line character varying (25)	sum bigint
1	Fashion accessories	902
2	Electronic accessories	971
3	Health and beauty	854
4	Food and beverages	952
5	Sports and travel	920
6	Home and lifestyle	911

Sales Analysis-4

```
219 -- 7. List the top 5 products by unit price.
220
221 SELECT product_line, unit_price
222 FROM walmart
223 ORDER BY unit_price limit 5;
224
```

Data Output Messages Notifications

SQL

	product_line character varying (25)	unit_price double precision
1	Health and beauty	10.08
2	Food and beverages	10.13
3	Health and beauty	10.16
4	Sports and travel	10.17
5	Fashion accessories	10.18

```
222
223 -- 8. Find sales transactions with a gross income greater than 30.
224
225 SELECT *
226 FROM walmart
227 WHERE gross_income > 30
228
```

Data Output Messages Notifications

SQL

	purchase_total double precision	purchase_date date	purchase_time time without time zone	payment character varying (15)	cogs double precision	gross_margin_percentage double precision	gross_income double precision	rating double precision	
1	2.21	634.38	2019-08-02	10:37:00	Ewallet	604.17	4.76	30.21	5.3
2	3.78	772.38	2019-02-24	11:38:00	Ewallet	735.6	4.76	36.78	8
3	3.69	749.49	2019-03-29	19:21:00	Cash	713.8	4.76	35.69	5.7
4	3.13	737.76	2019-01-28	12:43:00	Cash	702.63	4.76	35.13	4.5
5	3.51	703.75	2019-10-01	14:49:00	Cash	670.24	4.76	33.51	5.1
6	3.48	829.08	2019-04-03	20:39:00	Cash	789.6	4.76	39.48	7.6
7	1.32	867.62	2019-03-19	17:08:00	Ewallet	826.3	4.76	41.32	7.9
8	1.99	671.79	2019-03-02	10:19:00	Cash	639.8	4.76	31.99	9.5

Total rows: 137 of 137 Query complete 00:00:00.299

Ln 225, Col 1

Sales Analysis-5

```
229 -- 9. Retrieve sales transactions that occurred on weekends.
230
231 SELECT *, TO_CHAR(purchase_date, 'FMDay') AS weekday_name
232 FROM walmart
233 WHERE TRIM(TO_CHAR(purchase_date, 'Day')) = 'Saturday'
234        OR TRIM(TO_CHAR(purchase_date, 'Day')) = 'Sunday';
235
```

	purchase_date date	purchase_time time without time zone	payment character varying (15)	cogs double precision	gross_margin_percentage double precision	gross_income double precision	rating double precision	weekday_name text
1	10.22 2019-08-03	10:29:00	Cash	76.4	4.76	3.82	9.6	Saturday
2	10.53 2019-03-03	13:23:00	Credit card	324.31	4.76	16.22	7.4	Sunday
3	19.05 2019-01-27	20:33:00	Ewallet	465.76	4.76	23.29	8.4	Sunday
4	12.38 2019-02-24	11:38:00	Ewallet	735.6	4.76	36.78	8	Sunday
5	10.82 2019-06-02	18:07:00	Ewallet	57.92	4.76	2.9	4.5	Sunday
6	16.64 2019-11-03	11:03:00	Credit card	482.51	4.76	24.13	4.6	Sunday
7	14.63 2019-11-03	15:30:00	Ewallet	80.6	4.76	4.03	4.4	Sunday
8	11.44 2019-02-17	11:15:00	Ewallet	172.8	4.76	8.64	9.9	Sunday

Total rows: 284 of 284 Query complete 00:00:00.379 Ln 231, Col 1

```
236 -- 10. Calculate the total sales and gross income for each month.
237
238 v SELECT EXTRACT(MONTH FROM purchase_date) as Month,
239        ROUND(SUM(purchase_total::numeric),2) as total_sales,
240        ROUND(SUM(gross_income::numeric), 2) as gross_income
241 FROM walmart
242 GROUP BY 1
243 ORDER BY 1
```

	month numeric	total_sales numeric	gross_income numeric
1	1	86562.76	4122.22
2	2	63169.87	3008.21
3	3	72749.40	3464.40
4	4	7957.65	378.96
5	5	12798.72	609.49
6	6	9612.26	457.76
7	7	11500.72	547.66
8	8	13503.86	643.12
9	9	13767.31	655.61
10	10	9865.23	469.80
11	11	9618.37	458.02
12	12	11861.28	564.80

Total rows: 12 of 12 Query complete 00:00:00.306

Sales Analysis-6

```

247 -- 11. Find the number of sales transactions that occurred after 6 PM.
248
249 SELECT COUNT(*)
250 FROM walmart
251 WHERE (EXTRACT(HOUR FROM purchase_time)*60+EXTRACT(MINUTE FROM purchase_time)) > 1080;
252

```

Data Output Messages Notifications

SQL

	count bigint
1	279

```

244
245 -- 11. Find the number of sales transactions that occurred after 6 PM.
246
247 SELECT *
248 FROM walmart
249 WHERE (EXTRACT(HOUR FROM purchase_time)*60+EXTRACT(MINUTE FROM purchase_time)) > 1080;
250
251 -- 12. List the sales transactions that have a higher total than the average total of all transactions.
252

```

	product_line character varying (25)	unit_price double precision	quantity integer	tax_5percent double precision	purchase_total double precision	purchase_date date	purchase_time time without time zone	payment character varying (15)	cogs double precision
1	health and beauty	58.22	8	23.29	489.05	2019-01-27	20:33:00	Ewallet	465.76
2	electronic accessories	85.39	7	29.89	627.62	2019-03-25	18:30:00	Ewallet	597.73
3	fashion accessories	14.48	4	2.9	60.82	2019-06-02	18:07:00	Ewallet	57.92
4	health and beauty	71.38	10	35.69	749.49	2019-03-29	19:21:00	Cash	713.8
5	home and lifestyle	52.59	8	21.04	441.76	2019-03-22	19:20:00	Credit card	420.72
6	food and beverages	88.36	5	22.09	463.89	2019-01-25	19:48:00	Cash	441.8
7	fashion accessories	94.13	5	23.53	494.18	2019-02-25	19:39:00	Credit card	470.65
8	sports and travel	62.62	5	15.66	328.76	2019-10-03	19:15:00	Ewallet	313.1
9	home and lifestyle	86.72	1	4.34	91.06	2019-01-17	18:45:00	Ewallet	86.72
10	food and beverages	98.7	8	39.48	829.08	2019-04-03	20:39:00	Cash	789.6
11	health and beauty	15.37	2	1.54	32.28	2019-03-16	19:47:00	Cash	30.74
12	electronic accessories	85.98	8	34.39	722.23	2019-02-28	19:01:00	Cash	687.84

Total rows: 279 of 279 Query complete 00:00:00.332 Ln 247, Col 1

Sales Analysis-7

	min_purchase double precision	max_purchase double precision	avg_purchase_total numeric
1	10.68	1042.65	322.97

```
251 -- 12. List the sales transactions that have a higher total than the average total of all transactions.
252
253 SELECT *
254 FROM walmart
255 WHERE purchase_total > (SELECT
256                          AVG(purchase_total) AS avg_purchase_total
257                          FROM walmart)
258
```

Data Output Messages Notifications

id	ip_address	gender	product_line	unit_price	quantity	tax_5percent	purchase_total	purchase_date	purchase_time	product_category
	ip_address (10)	gender character varying (10)	product_line character varying (25)	unit_price double precision	quantity integer	tax_5percent double precision	purchase_total double precision	purchase_date date	purchase_time time without time zone	product_category
1		Female	Health and beauty	74.69	7	26.14	548.97	2019-05-01	13:08:00	Electronics
2		Male	Home and lifestyle	46.33	7	16.22	340.53	2019-03-03	13:23:00	Clothing
3		Male	Health and beauty	58.22	8	23.29	489.05	2019-01-27	20:33:00	Electronics
4		Male	Sports and travel	86.31	7	30.21	634.38	2019-08-02	10:37:00	Electronics
5		Male	Electronic accessories	85.39	7	29.89	627.62	2019-03-25	18:30:00	Electronics
6		Female	Electronic accessories	68.84	6	20.65	433.69	2019-02-25	14:36:00	Electronics
7		Female	Home and lifestyle	73.56	10	36.78	772.38	2019-02-24	11:38:00	Electronics
8		Male	Food and beverages	43.19	10	21.6	453.5	2019-07-02	16:48:00	Electronics
9		Female	Health and beauty	71.38	10	35.69	749.49	2019-03-29	19:21:00	Clothing
10		Female	Sports and travel	93.72	6	28.12	590.44	2019-01-15	16:19:00	Clothing
11		Female	Health and beauty	68.93	7	24.13	506.64	2019-11-03	11:03:00	Clothing
12		Male	Sports and travel	72.61	6	21.78	457.44	2019-01-01	10:39:00	Clothing

Total rows: 404 of 404 Query complete 00:00:00.309 Ln 253, Col 1

Sales Analysis-8

```
261 -- 13. Calculate the cumulative gross income for each branch by date.
262
263 SELECT branch,
264        purchase_date,
265        gross_income,
266        ROUND(SUM(gross_income) OVER (PARTITION BY branch ORDER BY purchase_date, gross_income)::numeric, 2) AS cumulative_income
267 FROM walmart
268 ORDER BY branch, purchase_date;
```

Data Output Messages Notifications

SQL

	branch character varying (2)	purchase_date date	gross_income double precision	cumulative_income numeric
1	A	2019-01-01	5.41	5.41
2	A	2019-01-01	19.04	24.45
3	A	2019-01-01	21.78	46.23
4	A	2019-01-01	29.58	75.81
5	A	2019-01-01	37.11	112.92
6	A	2019-01-02	3.56	116.48
7	A	2019-01-02	15.54	132.02
8	A	2019-01-02	24.33	156.35
9	A	2019-01-03	3.47	159.82
10	A	2019-01-03	4.46	164.28
11	A	2019-01-03	22.74	187.02
12	A	2019-01-13	7.04	194.06
13	A	2019-01-13	12.15	206.21

Total rows: 1000 of 1000 Query complete 00:00:00.318

Ln 268, Col 3:

Sales Analysis-9

```
268 -- 14. Find the total cogs for each customer type in each city.
269
270 v SELECT city,
271         customer_type,
272         ROUND(SUM(cogs::numeric), 2) as total_cogs
273 FROM walmart
274 GROUP BY 1, 2
275 ORDER BY 1, 2
```

Data Output Messages Notifications

	city character varying (15)	customer_type character varying (10)	total_cogs numeric
1	Mandalay	Member	51147.32
2	Mandalay	Normal	49993.32
3	Naypyitaw	Member	54172.65
4	Naypyitaw	Normal	51130.88
5	Yangon	Member	51083.31
6	Yangon	Normal	50059.90



ANY QUESTIONS?